```
require "time"
Time.strptime("2023-2-10", "%Y-%m-%d") # => 2023-02-10 00:00:00 -0600
Time.strptime("2023-2-10", "%Y-%d-%m") # => 2023-10-02 00:00:00 -0500
```

Another way to create a Time instance is using the multi-argument form of Time.new. The multi-argument form is:

Time.new(year, month = 1, day_of_month = 1, hour = 0, minute = 0, second = 0, time_zone = local, in: nil)

All the positional arguments other than year are optional, and all the values can be of type Integer, Float, Rational, or String instances that can be converted to integers. In addition:

- The month value range is 1 to 12, or it can be the three-letter English abbreviation of the month (case-insensitive).
- The day of month is between 1 and 31. If the month in question has fewer than 31 days, then Ruby will push forward into the next month, so Time.new(2023, 2, 31) returns 2023-03-03 00:00:00 -0600.
- The hour ranges from 0 to 23, but you can use 24 if and only if the minute and second are zero.
- Minute ranges from 0 to 59.
- Second ranges from 0 to 60, where 60 indicates a leap second. The second can also be a float or rational number.

If any of the values are out of range, Ruby throws an ArgumentError.

The time zone can be specified via the last positional argument or the in: keyword argument using the same timezone specifiers we saw earlier withTime.now.

A rarely used from of Time.new takes in a string in YYYY-MM-DD HH:MM:SS format and returns that exact time. This version takes a keyword argument, precision, which limits the number of decimal places kept for the seconds.

A couple of variations on Time.new specify the time zone:

- Time.gm or Time.utc, which returns the time in UTC.
- Time.local or Time.mktime, which return the time in the local timezone.

All of these methods take up to seven positional arguments that match the arguments to Time.now but with an additional argument for microseconds. Alternately, the methods are set up such that if they are called with exactly ten arguments, they assume the arguments are in order as if they were generated by Time#to_a. So x = Time.now; Time.gm(*x.to_a) will work as expected even though the order of the result of to_a does not match the order of positional arguments in Time.now.

The method Time.at returns a new time object given a number of seconds since the beginning of the Unix Epoch on Jan 1, 1970. It takes an optional in: keyword argument to specify the time zone of the result using the time zone specifiers discussed earlier. An optional second positional argument can specify subseconds, and an optional third argument specifies the unit of the subseconds. The unit can be :millisecond, :microsecond, or :nanosecond. The default is :microsecond.